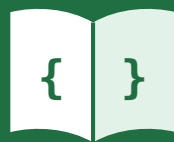

Module 00 — Setup and Modern Java (21 → 27)

Exercises

Spring Boot Edu



2026-09-23

Contents

How to Work	2
Exercise 1 — Customer Discounts (Easy)	2
Exercise 2 — Parallel Stock Lookup (Medium)	2
Exercise 3 — Order State Machine (Hard)	3
Bonus Challenge (Optional)	4

How to Work

1. The starter code is in `modules/00-setup-modern-java/exercise/`. Find the TODO comments.
2. Every exercise already has tests, and they stay **red** until you solve it.
3. Run the tests:

```
./mvnw -Pexercises -pl modules/00-setup-modern-java/exercise test
```

4. The exercise is done when all tests are green.
5. If you get stuck, read the hints first, then the solution in `modules/00-setup-modern-java/solution/`.

★ Tip

To run the tests of a single exercise: `-Dtest=Exercise1Test`

Exercise 1 — Customer Discounts (Easy)

Goal: make decisions with pattern matching and when guards on a sealed type.

The bookstore has three kinds of customers: `Regular`, `Student(university)` and `Member(years)`. `Customer` is sealed, so no other kind can exist.

Tasks (`exercise1.DiscountPolicy`):

- TODO 1a — `rateFor(customer)`: return 0 for `Regular`, 10 for `Student` and 5 for `Member`. From 2 years of membership it is 15, from 5 years 20.
- TODO 1b — `priceFor(customer, price)`: apply the discount to the price and round to 2 decimals (`HALF_UP`).

Hints:

- Lesson sections 3.2 and 3.3: `case Member(int years) when years >= 5 -> ...`
- when guards are tried top to bottom. Put the most specific case first.
- Do not add a default branch. The compiler checks exhaustiveness for you.

Acceptance criteria: all 4 tests in `Exercise1Test` pass.

Estimated time: 20 minutes

Exercise 2 — Parallel Stock Lookup (Medium)

Goal: run many calls to a slow service in parallel on virtual threads.

`StockClient.stockOf(isbn)` is a remote call that blocks on every invocation. Asking for the stock of 200 books one after another takes 40 seconds. The target is under 3 seconds.

Tasks (exercise2.StockChecker):

- TODO 2a — run `client.stockOf(isbn)` for every ISBN on its own virtual thread.
- TODO 2b — return the results in the **same order** as the input list.
- TODO 2c — turn the checked exceptions of `Future.get()` into an `IllegalStateException`. Do not forget to restore the interrupt flag on `InterruptedException`.

Hints:

- Lesson section 3.5: `Executors.newVirtualThreadPerTaskExecutor()` with try-with-resources.
- `LinkedHashMap` keeps insertion order.
- The test also checks that the calls really run on virtual threads.

Acceptance criteria: both tests in `Exercise2Test` pass.

Estimated time: 30 minutes

Exercise 3 — Order State Machine (Hard)

Goal: write a rule-based state machine by combining two sealed hierarchies (state × event) with nested record patterns.

An order follows `New → Paid → Shipped → Delivered`. Orders in the `New` and `Paid` states can be cancelled (`Cancel`). Every other transition is invalid.

Tasks (exercise3.OrderStateMachine):

- TODO 3a — implement the valid transitions:

State	Event	New state
New	<code>Pay(amount), amount > 0</code>	<code>Paid(amount)</code>
Paid	<code>Ship(trackingNumber)</code>	<code>Shipped(trackingNumber)</code>
Shipped	<code>Deliver</code>	<code>Delivered</code>
New or Paid	<code>Cancel(reason)</code>	<code>Cancelled(reason)</code>

- TODO 3b — in every other case throw `IllegalStateException("Cannot <Event> when <State>")`, e.g. `Cannot Ship when New`.

Hints:

- Combine state and event into one value: `private record Transition(OrderState state, OrderEvent event)`.

- Then switch (`new Transition(state, event)`) with nested patterns: `case Transition(New _, Pay(BigDecimal amount)) when amount.signum() > 0 -> ...`
- For the class name: `event.getClass().getSimpleName()`.

Acceptance criteria: all 4 tests in `Exercise3Test` pass.

Estimated time: 45 minutes

Bonus Challenge (Optional)

Rewrite Exercise 2 with structured concurrency (lesson section 3.10). Put your code under `src/preview/java` and run it with `-Ppreview`. Write a test showing that the other calls are cancelled when one fails.